Session 2: Security goals

Slides prepared by UMBC Protocol Analysis Lab

Last edited July 28, 2026

What is a security goal?

What is a security goal?

A *security goal* formalizes our notion of “secure” by defining an objective for protecting protocol information.

What is a security goal?

A *security goal* formalizes our notion of “secure” by defining an objective for protecting protocol information.

Not every protocol has the same requirement for security.
Common Protocol Security Goals

What is a security goal?

A *security goal* formalizes our notion of "secure" by defining an objective for protecting protocol information.

Not every protocol has the same requirement for security.
Common Protocol Security Goals

- **Confidentiality:** No unauthorized party is able to access the information sent. For example, an eavesdropping adversary does not learn where Alice and Bob wish to meet.

What is a security goal?

A *security goal* formalizes our notion of “secure” by defining an objective for protecting protocol information.

Not every protocol has the same requirement for security.
Common Protocol Security Goals

- **Confidentiality:** No unauthorized party is able to access the information sent. For example, an eavesdropping adversary does not learn where Alice and Bob wish to meet.
- **Agreement:** Both parties agree on some set of data values in the protocol. For example, Alice and Bob agree on a meeting location.

Other common security goals

1. Privacy
2. Anonymity
3. Perfect Forward Secrecy
4. Non-Repudiation

Needham Schroeder

Designing a meeting protocol, and beyond

Strand spaces

A *strand space* is a formalism used to determine the security of a network protocol

- Fábrega, F. J. T., Herzog, J. C., & Guttman, J. D. (1998, May). Strand spaces: Why is a security protocol correct?
- Represent honest communicants and intruders as "strands"
- Parallel / sequential composability.

Example of a strand

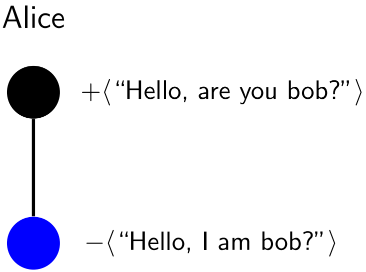
Alice

Example of a strand

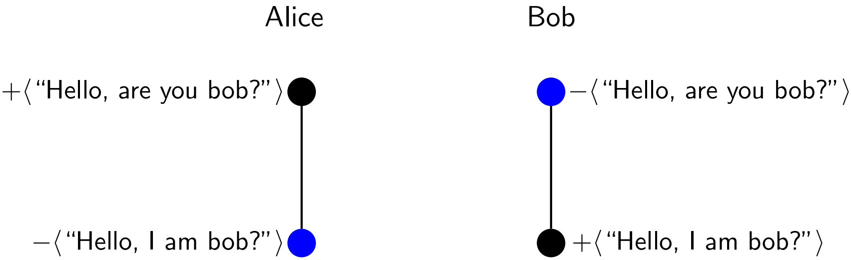
Alice

$+\langle$ "Hello, are you bob?" $\rangle$

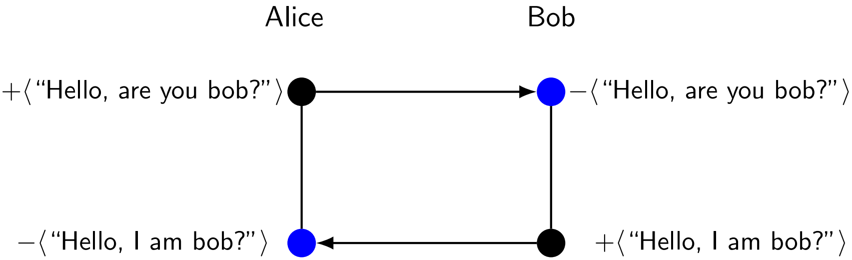
Example of a strand



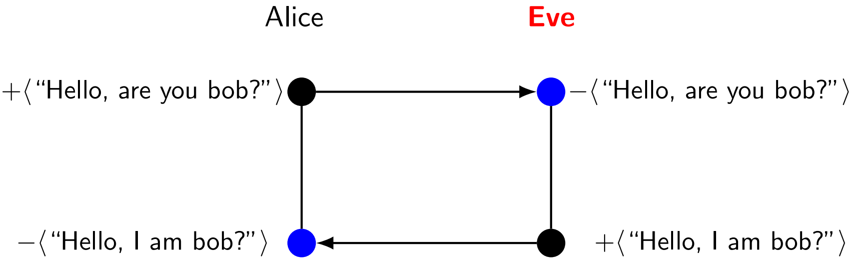
Strand space



A completed protocol?



Strands can lie



CPSA Penetrator Strands

CPSA Penetrator Strands

M. Create Primitive: $\langle +p \rangle$ where $p \in P$ where P is the set of all basic values.
C. Concatenation: $\langle -g, -h, +gh \rangle$ where $g, h, gh \in G$
S. Separation into components: $\langle -gh, +g, +h \rangle$
E. Encryption $\langle -K, -h, +\{h\}_K \rangle$
D. Decryption $\langle -K, -\{h\}_K, +h \rangle$

1. $\langle +m \rangle$ emit a message from the network.
2. $\langle -m \rangle$ extract a message from the network.
3. $\langle m_1 m_2 \rangle$ concatenate two messages.
4. $\langle \{m\}_K \rangle$ encrypt a message m under key K.

CPSA Penetrator Strands

M. Create Primitive: $\langle +p \rangle$ where $p \in P$ where P is the set of all basic values.

C. Concatenation: $\langle -g, -h, +gh \rangle$ where $g, h, gh \in G$

S. Separation into components: $\langle -gh, +g, +h \rangle$

E. Encryption $\langle -K, -h, +\{h\}_K \rangle$

D. Decryption $\langle -K, -\{h\}_K, +h \rangle$

1. $\langle +m \rangle$ emit a message from the network.
2. $\langle -m \rangle$ extract a message from the network.
3. $\langle m_1 m_2 \rangle$ concatenate two messages.
4. $\langle \{m\}_K \rangle$ encrypt a message m under key K.

These are identical to the capabilities of a *regular* user.

CPSA Penetrator Strands

M. Create Primitive: $\langle +p \rangle$ where $p \in P$ where P is the set of all basic values.
C. Concatenation: $\langle -g, -h, +gh \rangle$ where $g, h, gh \in G$
S. Separation into components: $\langle -gh, +g, +h \rangle$
E. Encryption $\langle -K, -h, +\{h\}_K \rangle$
D. Decryption $\langle -K, -\{h\}_K, +h \rangle$

1. $\langle +m \rangle$ emit a message from the network.
2. $\langle -m \rangle$ extract a message from the network.
3. $\langle m_1 m_2 \rangle$ concatenate two messages.
4. $\langle \{m\}_K \rangle$ encrypt a message m under key K.

These are identical to the capabilities of a *regular* user.
What does that tell us about the adversary?

Needham Schroeder

Alice Bob

= Reception = Transmission

Needham Schroeder

Alice Bob

$$\text{Alice} \xrightarrow{enc(N_a, Alice, pubk_{Bob})} \text{Bob}$$

● = Reception ● = Transmission

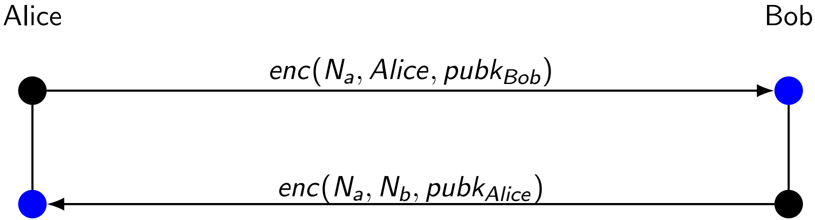
Needham Schroeder

Alice $\xrightarrow{enc(N_a, Alice, pubk_{Bob})}$ Bob

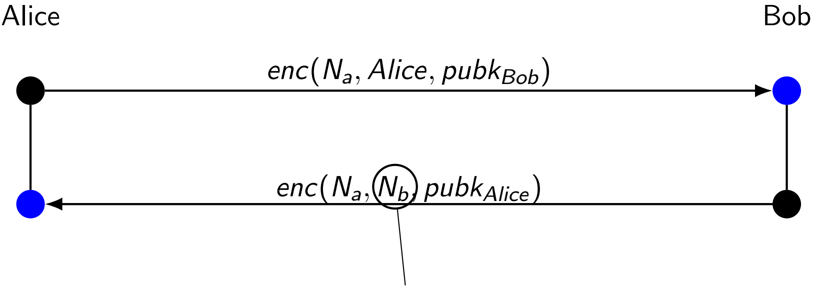
nonce: a *fresh*, cryptographic random number generated by Alice.

= Reception = Transmission

Needham Schroeder

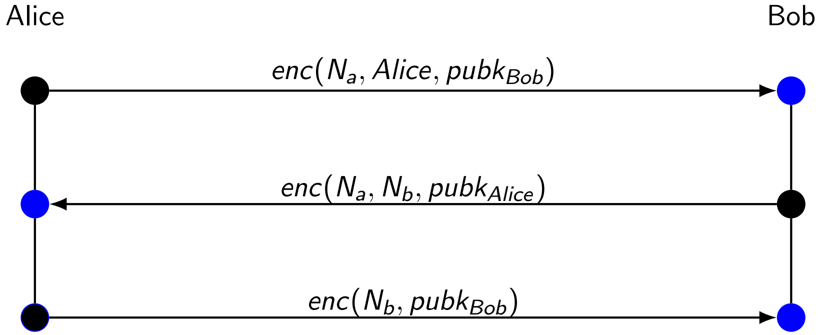


Needham Schroeder



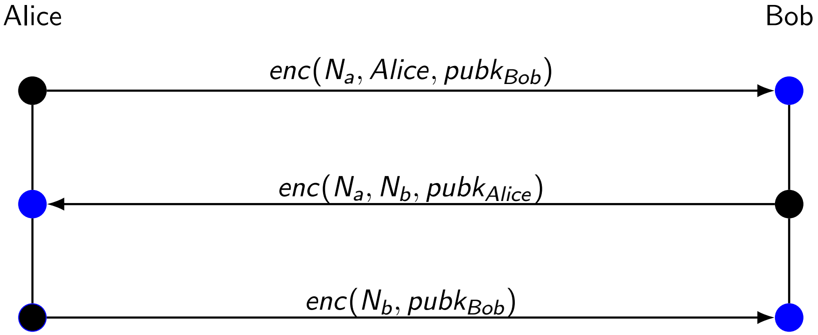
nonce: a *fresh*, cryptographic random number generated by Bob.

Needham Schroeder



= Reception = Transmission

Needham Schroeder



Is this protocol secure against the Dolev-Yao Adversary?

Cryptographic Protocol Shapes Analyzer

- Tool for analyzing and developing secure protocols.
- Enumerates all possible executions of a protocol.
- Assumes a Dolev-Yao adversary.
- Developed by MITRE, available at: https://github.com/mitre/cpsa

Analyze Needham-Schroeder

Define protocol *roles* and *skeletons*.
Run CPSA to produce *shapes*.
Analyze *shapes* for expected and unexpected behavior.

CPSA Exploration

CPSA Exploration

Now that you've gotten your feet wet with CPSA, take a look at the documentation!

We have an exploration to guide you through the manual. We encourage you to read "around" the areas where the answers are as well, and try out some of the examples in the manual!

Survey

Today's survey can be found at the QR code below. The link will also be sent out in a discord announcement later today.